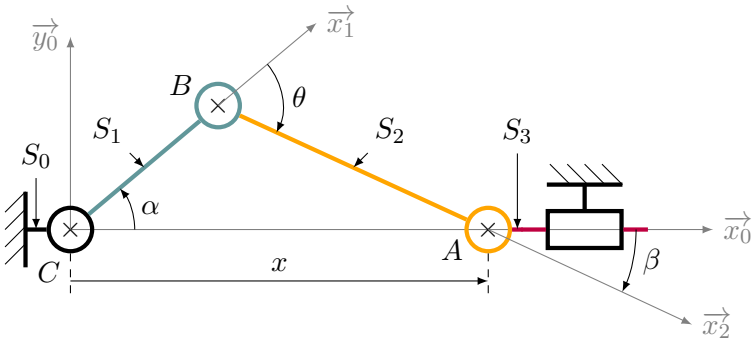
Dérivation numérique, fichiers de données
ÉTUDE CINÉMATIQUE D'UN COMPRESSEUR

1 Présentation et modélisation



Soit le système bielle manivelle d'un compresseur dont le schéma cinématique plan est donné ci-dessous. La longueur de la manivelle **1** (L_1) et de la bielle **2** (L_2) sont des caractéristiques géométriques connues et invariables.



Les paramètres α, β et x sont des paramètres de position représentatifs des mouvements du système.

- le paramètre d'entrée est α, il traduit la rotation de la manivelle **1** par rapport à **0** autour de l'axe $(O, \overrightarrow{z_0})$;
- le paramètre de sortie est x, il traduit la translation du piston **3** par rapport à **0** suivant de l'axe $(O, \overrightarrow{x_0})$;
- le paramètre β est un paramètre intermédiaire qui traduit la rotation de la bielle **2** par rapport à **0** autour de l'axe $(B, \overrightarrow{z_0})$.

CC BY NC SA

La fermeture géométrique donne la loi entrée-sortie suivante :

$$x = L_1 \cos\alpha + \sqrt{L_2^2 - (L_1 \sin\alpha)^2}$$

1.1 Étude de l'évolution de la sortie en fonction de l'entrée

On considère pour cette partie que $L_1 = 20\,\text{mm}$ et $L_2 = 2L_1$. De plus la vitesse de rotation du moteur entrainant le compresseur est constante ($\dot{\alpha} = \text{cste}$) et vaut 1500 tours par minute.

1.1.1 Évolution de la position du piston

On cherche à tracer en fonction de α la position du piston repérée par x. **On s'obligera volontairement à ne pas utiliser la bibliothèque** `numpy`.

Activité 1

- Écrire une fonction `xpiston(a,e,L)` qui prend en arguments d'entrées deux flottants `e` et `L` représentant respectivement la longueur de l'excentrique et de la bielle, ainsi qu'une liste `a` de valeurs de la position angulaire. La fonction devra renvoyer une liste de même taille que `a` et elle représentera les valeurs des positions du piston.

```
1  # Import des bibliothèques nécessaires
2  from math import cos, sin, sqrt, pi
3
4  import matplotlib.pyplot as plt
5
6  # On saisit les paramètres de l'étude
7  L1 = 20 ; L2 = 2*L1
8  Nmot=1500; wmot=Nmot*pi/30
9
10 # Définition de la fonction xpiston
11 def xpiston(a,e,L):
12     """L'argument a est une liste
13     e et L sont des flottants
14     la sortie est une liste
15     """
16     y=[] #création d'une liste vide
17     for i in range(len(a)):
18         q=a[i]
19         y.append(e*cos(q) + sqrt(L**2-(e*sin(q))**2))
20     return y
```

Activité 2

- Tracer sur une figure 1 l'évolution de x pour 4 périodes en fonction de α. On prendra une précision de 1°. On prendra soin de légender la figure obtenue et l'axe des abscisses sera dimensionné en degrés, tandis que celui des ordonnées sera en mm.

```python
# Création d'une liste d'entrée de 4 périodes
a_deg=[float(k) for k in range(int(4*360))] # en degrés
a_rad=[]; # en radians
for k in a_deg:
    a_rad.append(k*pi/180.)

# Génération de la liste "position piston"
x=xpiston(a_rad,L1,L2)

plt.figure(1)
plt.plot(a_deg,x,color="r")
plt.title("Evolution sur 4 périodes de la position du piston")
plt.ylabel('x (mm)')
plt.xlabel("alpha (deg)")
```

1.1.2 Analyse de la vitesse

Dans cette deuxième partie, on s'intéresse aux aspects cinématiques de la transformation de mouvement.

Activité 3

- Quel est le temps mis par le compresseur pour décrire 4 périodes ?

```python
# Calcul du temps mis par le compresseur pour faire 4 aller-retours
# Durée d'une période :
T0 = 2*pi/wmot
print("La durée de 4 périodes est {} s".format(4*T0))
```

Activité 4

- Toujours pour 4 périodes et en considérant la vitesse de rotation constante du vilebrequin de 1500 tours par minute, tracer l'évolution du piston en fonction du temps sur une figure 2. On prendra une résolution identique au tracé précédent. Quelle est alors cette résolution (période d'échantillonnage) ?

```python
# Création d'une liste "temps" de même taille
t=[k/wmot for k in a_rad]

plt.figure(2)
plt.plot(t,x,color="r")
plt.title("Evolution sur 4 periodes de la position du piston")
plt.ylabel('x (mm)')
plt.xlabel("t (s)")

#"Période d'échantillonnage"
Te=t[1];
print("\nLa période Te = {}s".format(Te))
```

Nous allons maintenant calculer par dérivation numérique la vitesse de déplacement du piston en fonction du temps.

Activité 5

- Écrire une fonction `vpiston(x,t)` qui prend en arguments d'entrée une liste `x` de positions du piston et une liste `t` de même taille regroupant l'ensemble des dates liées aux positions successives du piston. Cette fonction retournera une liste contenant les vitesses successives pour chaque date précédemment définie.

```
# Fonction permettant de calculer la vitesse
def vpiston(x,t):
    v=[]
    for i in range(len(x)-1):
        dx=x[i+1]-x[i]
        dt=t[i+1]-t[i]
        v.append(dx/dt)
    return v

# Détermination de la liste des vitesses
v=vpiston(x,t)  # ATTENTION : UN ELEMENT DE MOINS !
```

Activité 6

- Tracer sur une figure 3, deux graphes côte à côte (se renseigner sur la fonction `subplot`). Celui de gauche donnera l'évolution de la vitesse en fonction du temps, et celui de droite l'évolution de la vitesse en fonction de la position x du piston. Pour simplifier, on gardera l'unité mm/s pour la vitesse.

```
plt.figure(3)
plt.subplot(1,2,1)
plt.plot(t[:-1],v)
plt.title("Vitesse du piston sur 4 périodes")
plt.ylabel('v (mm/s)')
plt.xlabel("t (s)")
plt.subplot(1,2,2)
plt.plot(x[:-1],v)
plt.title("Vitesse du piston sur 4 périodes")
plt.ylabel('v (mm/s)')
plt.xlabel("x (mm)")
```

1.1.3 Vitesse théorique et vitesse calculée par dérivation numérique

Afin de vérifier si le calcul effectué numériquement renvoie une vitesse correcte, on se propose de la comparer avec la vitesse théorique du piston.

Activité 7

- Donner l'expression de la vitesse $\dot{x}$ en fonction de α et $\dot{\alpha}$.

$$\dot{x} = -L_1 \dot{\alpha} \sin\alpha \left(1 + \frac{L_1 \cos\alpha}{\sqrt{L_2^2 - (L_1 \sin\alpha)^2}}\right)$$

Activité 8

- Écrire une fonction `vpiston_th(a,w,e,L)` qui retourne une liste de valeurs de vitesses de translation du piston calculées à partir des arguments d'entrée suivants :
  - ◇ liste `a` de position angulaire ;
  - ◇ un flottant w correspondant à la vitesse de rotation en radians par seconde du vilebrequin, supposée constante ;
  - ◇ et `e` et `L` représentant respectivement l'excentrique et la longueur de la bielle.

```
# Définition de la fonction retournant la liste "vitesse théorique"
def vpiston_th(a,w,e,L):
    # On décompose les calculs pour faciliter la compréhension
    y=[]
    for i in range(len(a)):
        q=a[i]
        N=e*cos(q)
        D=sqrt(L**2-(e*sin(q))**2)
        y.append(-e*w*sin(q)*(1+N/D))
    return y

vth=vpiston_th(a_rad,wmot,L1,L2)
```

La vitesse théorique peut alors être tracée et afin de vérifier que la méthode de dérivation numérique est satisfaisante, nous allons comparer ces deux vitesses.

Activité 9

- Sur une nouvelle fenêtre graphique (figure 4), superposer les vitesses théorique et calculée. On tracera la vitesse en fonction du temps (toujours sur 4 périodes). La vitesse théorique sera tracée en ligne continue bleue, tandis que la vitesse calculée le sera en rouge.

```
plt.figure(4)
plt.plot(t,vth,"b-",label="V theorique")
plt.plot(t[:-1],v,"r+",label="V calculee")
plt.ylabel('v (mm/s)')
plt.xlabel("t (s)")
```

On souhaite maintenant observer les erreurs de calcul engendrées par la méthode numérique. Nous allons donc évaluer l'écart entre la vitesse théorique et la vitesse calculée numériquement.

Activité 10

- Mettre en œuvre le tracé de l'écart « vitesse théorique – vitesse numérique » sur une nouvelle figure. On tracera cet écart en fonction du temps.

```
# Calcul des écarts
ecart=[]
for k in range(len(v)):
    ecart.append(vth[k]-v[k])

# Courbe d'écarts
plt.figure(5)
plt.plot(t[:-1],ecart,'r-')
plt.ylabel('v (mm/s)')
plt.xlabel("t (s)")
```

Activité 11

- Que constate-t-on ?
- Que faudrait-il faire pour améliorer la précision ?

1.1.4 Recherche de la vitesse maximale

On se basera sur la vitesse calculée numériquement dans cette partie. On souhaite rechercher la position angulaire pour laquelle la vitesse maximale.

Activité 12

- Mettre en œuvre une méthode pour trouver la (ou les) valeurs de l'angle α pour la(les)quelle(s) la vitesse est maximale. On donnera le (ou les) angle(s) dans l'intervalle $[0\,;\,2\pi]$.

```
#Recherche de la vitesse maximale (compression) et de la position angulaire correspondante
vmax=0; posa_vmax=0;
for i in range(int(len(v)/4)): #On demande de rechercher sur une seule période
    if v[i]>vmax:
        vmax=v[i]
        posa_vmax=a_deg[i]

print("\nLa valeur maximale de la vitesse du piston en phase de compression est : {} mm/s".
    format(vmax))
print("Elle est obtenue pour un angle d'entrée de {} degrés".format(posa_vmax))

#Recherche de la vitesse maximale (compression) et de la position angulaire correspondante
```

```
vmin=0; posa_vmin=0;
for i in range(int(len(v)/4)): #On demande de rechercher sur une seule période
    if v[i]<vmin:
        vmin=v[i]
        posa_vmin=a_deg[i]

print("\nLa valeur maximale de la vitesse du piston en phase de detente est : {} mm/s".format(
    abs(vmin)))
print("Elle est obtenue pour un angle d'entrée de {} degrés".format(posa_vmin))
```

1.2 Identification à partir de résultats expérimentaux

Un relevé expérimental a permis de mesurer la position du piston en fonction de l'angle du vilebrequin. Pour simplifier le travail, la correspondance entre l'angle et la position mesurée est la même que sur la modélisation prise au départ.

1.2.1 Visualisation des résultats expérimentaux

Activité 13

- Ouvrir le fichier `compresseur.csv` ;
- Observer sa structure, et sous Python, créer deux listes – l'une contenant les valeurs de position angulaire que l'on nommera angle et une deuxième contenant les valeurs mesurées de position du piston que l'on nommera `xmes`.

```
from math import cos, sin, sqrt, pi

import matplotlib.pyplot as plt

# Ouverture du fichier et remplissage des listes
fi=open("Exp.csv")

angle=[]; xmes=[]
for li in fi:
    if li[0] not in "abcdefghijklmnopqrstuvwxyz":
        li_sep=[]
        li_sep=li.split(";")
        angle.append(float(li_sep[0]))
        xmes.append(float(li_sep[1][:-1]))
fi.close()
```

Activité 14

- Tracer alors la position mesurée en fonction de l'angle du vilebrequin.

```
# Tracé des la figure
plt.figure(1)
plt.plot(angle,xmes,"b*")
plt.title("Resultats experimentaux")
plt.ylabel('x_mesure (mm)')
plt.xlabel("alpha (deg)")
```

1.2.2 Identification de l'excentrique et de la longueur de la bielle

On souhaite dans cette partie retrouver les valeurs de l'excentrique et de la longueur de la bielle, à partir des relevés expérimentaux étudiés ici. Nous allons tout d'abord déterminer la valeur L_1.

Activité 15

- En remarquant que l'amplitude du mouvement est égal à deux fois la longueur de l'excentrique, identifier numériquement la valeur de L_1 pour notre cas expérimental.

```
# Recherche des valeurs mini et maxi
xmin=xmes[0]; xmax=xmes[0]
for k in xmes:
    if k>xmax:
        xmax=k
    if k<xmin:
        xmin=k

# Détermination de l'excentrique L1
L1=(xmax-xmin)/2.
```

Activité 16

- Exprimer L_2 en fonction de L_1, α et x.

Activité 17

- À l'aide d'une boucle, calculer la grandeur L_2 pour chacune des valeurs expérimentales. Les valeurs de L_2 trouvées pour chaque point expérimental pourront être stockées dans une liste `liste_L2`. En déduire la valeur moyenne de L_2.

```
# On passe les angles en radians !
ang_rad=[]
for k in angle:
```

```
    ang_rad.append(k*pi/180)

liste_L2=[]
for i in range(len(xmes)):
    L2=sqrt((xmes[i]-L1*cos(ang_rad[i]))**2 + (L1*sin((ang_rad[i]))**2))
    liste_L2.append(L2)

L2=0
for k in liste_L2:
    L2+=k/len(liste_L2)
```

Pour la question suivante, on réutilisera la fonction `xpiston` définie dans la première partie.

Activité 18

- Les paramètres L_1 et L_2 étant déterminés, tracer sur la figure déjà ouverte la courbe représentant le modèle ainsi identifié ;
- Conclure sur l'identification des paramètres.

```
#Definition de la fonction xpiston
def xpiston(a,e,L):
    """L'argument a est une liste
    e et L sont des flottants
    la sortie est une liste
    """
    y=[] #création d'une liste vide
    for i in range(len(a)):
        q=a[i]
        y.append(e*cos(q) + sqrt(L**2-(e*sin(q))**2))
    return y

a_deg=[float(k) for k in range(int(360))] #en degrés
a_rad=[]; # en radians
for k in a_deg:
    a_rad.append(k*pi/180.)

x_mod=xpiston(a_rad,L1,L2)

plt.plot(a_deg,x_mod,"r-")

plt.show()
```